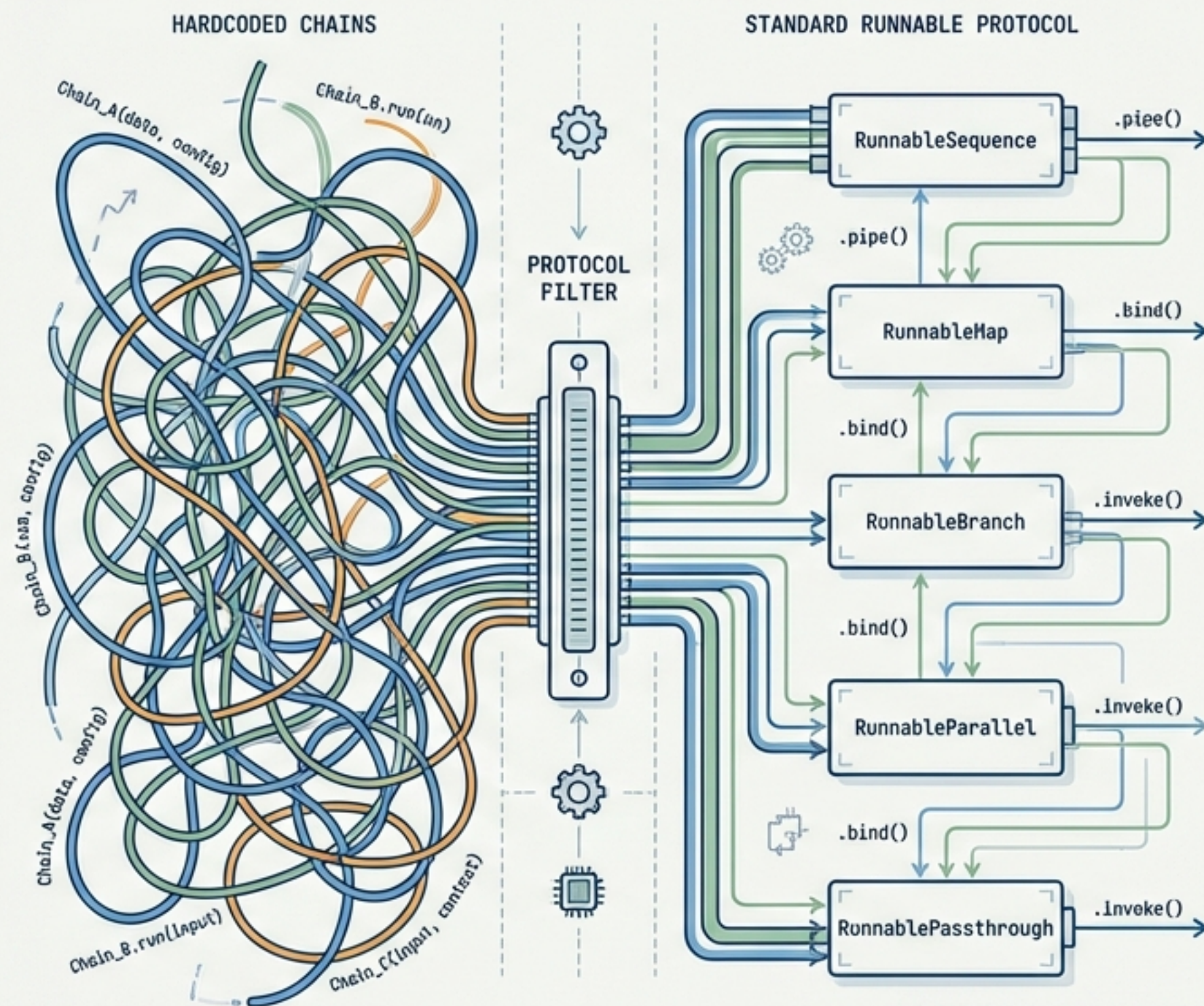


# Decoding LangChain Runnables

The architectural evolution from hardcoded Chains to the standard Runnable Protocol.

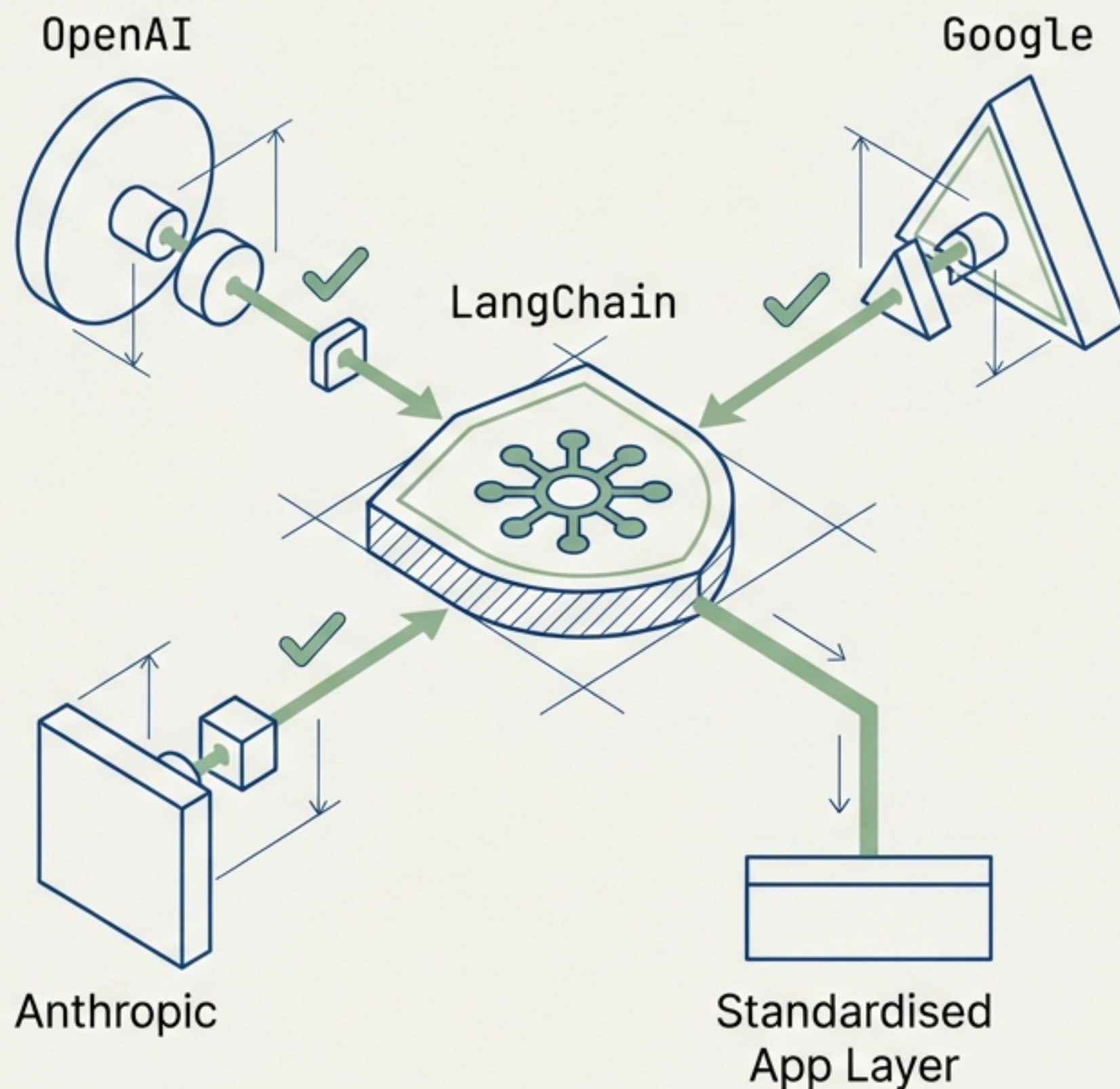


Based on the technical deep-dive by Nitesh (CampusX)

# The Origin Story: Post-ChatGPT API Fragmentation

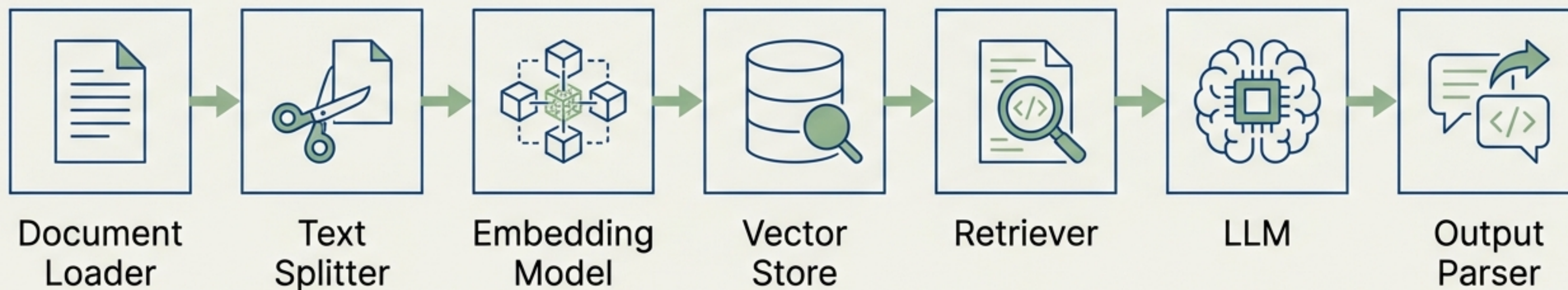
Late 2022: Following the release of ChatGPT, developers rushed to build LLM applications. However, every provider released APIs with **completely different structures**.

LangChain's First Value Prop: A "Universal Translator". The team created wrapper components to standardise interactions across different models, allowing developers to swap models with **minimal code changes**.



# Beyond the Model: The Component Ecosystem

Building an app requires a full data pipeline, not just an LLM interaction. LangChain expanded to provide helper classes for every stage.



# The First Abstraction: Birth of “Chains”

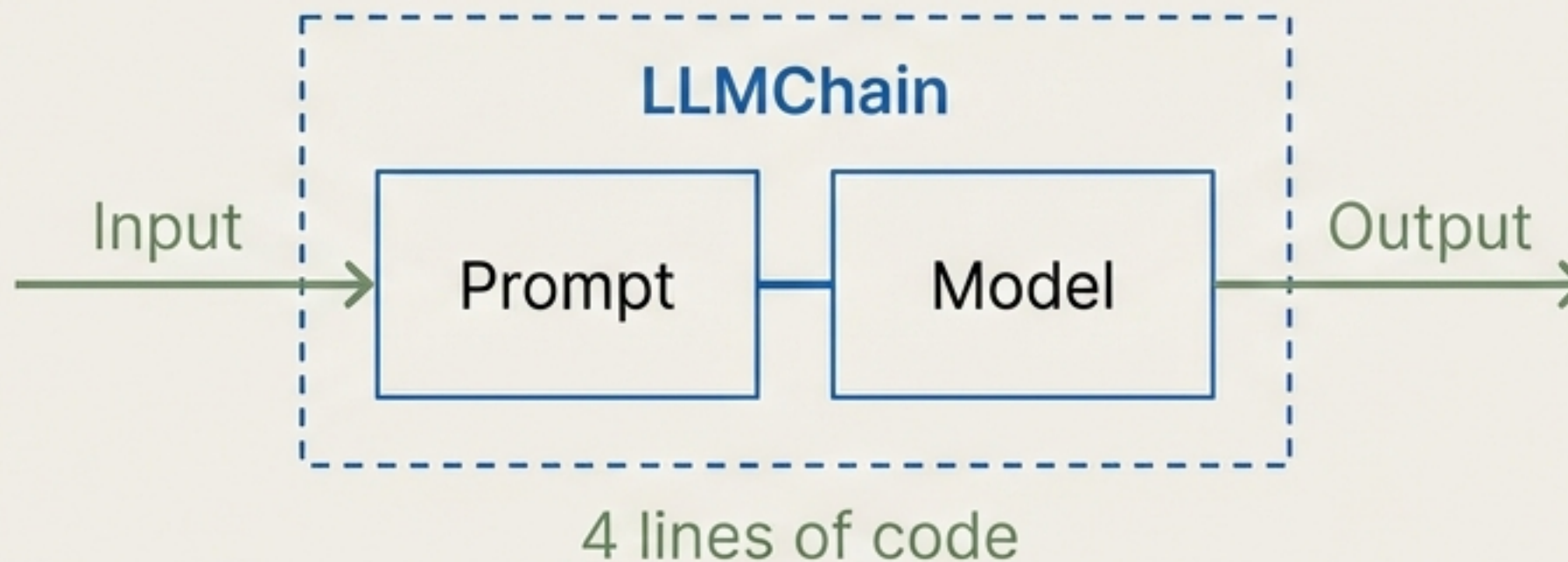
Developers were manually coding repetitive connections. The "**Eureka**" moment was to create **built-in functions** to automate these workflows.

## Manual Connection



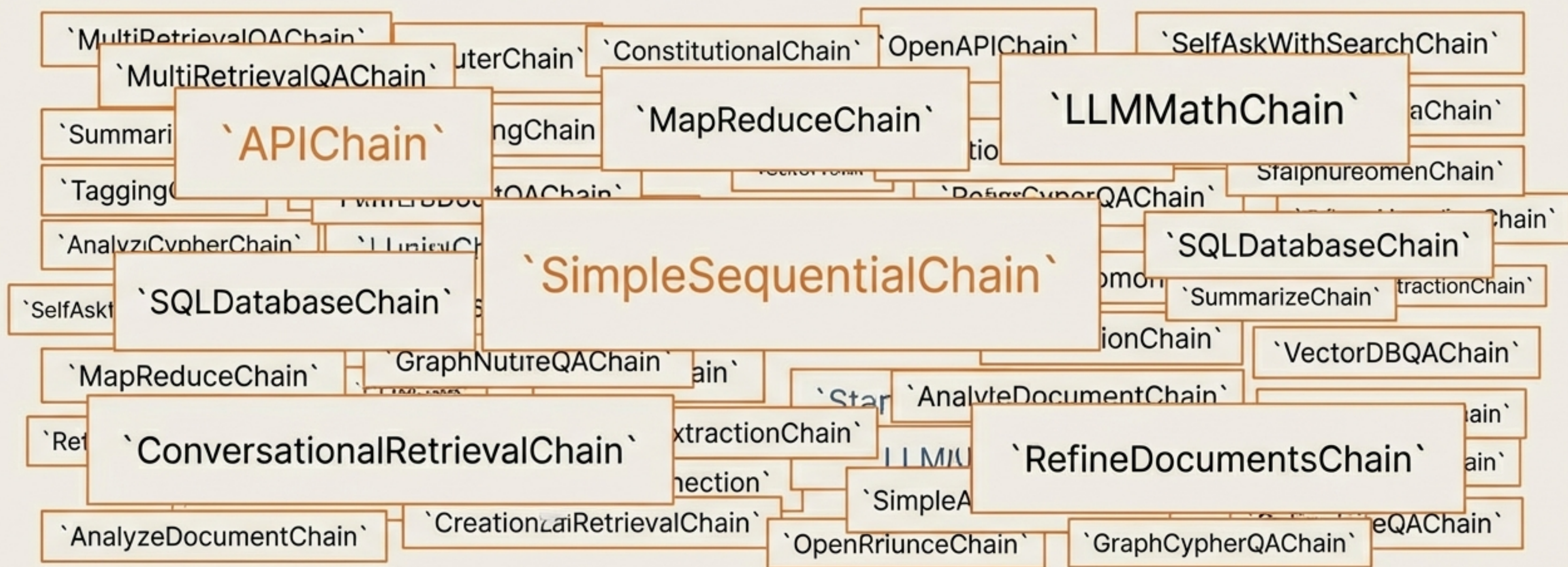
Key Artifacts:  
`LLMChain`  
(Prompt + Model),  
`RetrievalQAChain`  
(Retriever + Prompt + Model)

## The Chain Solution



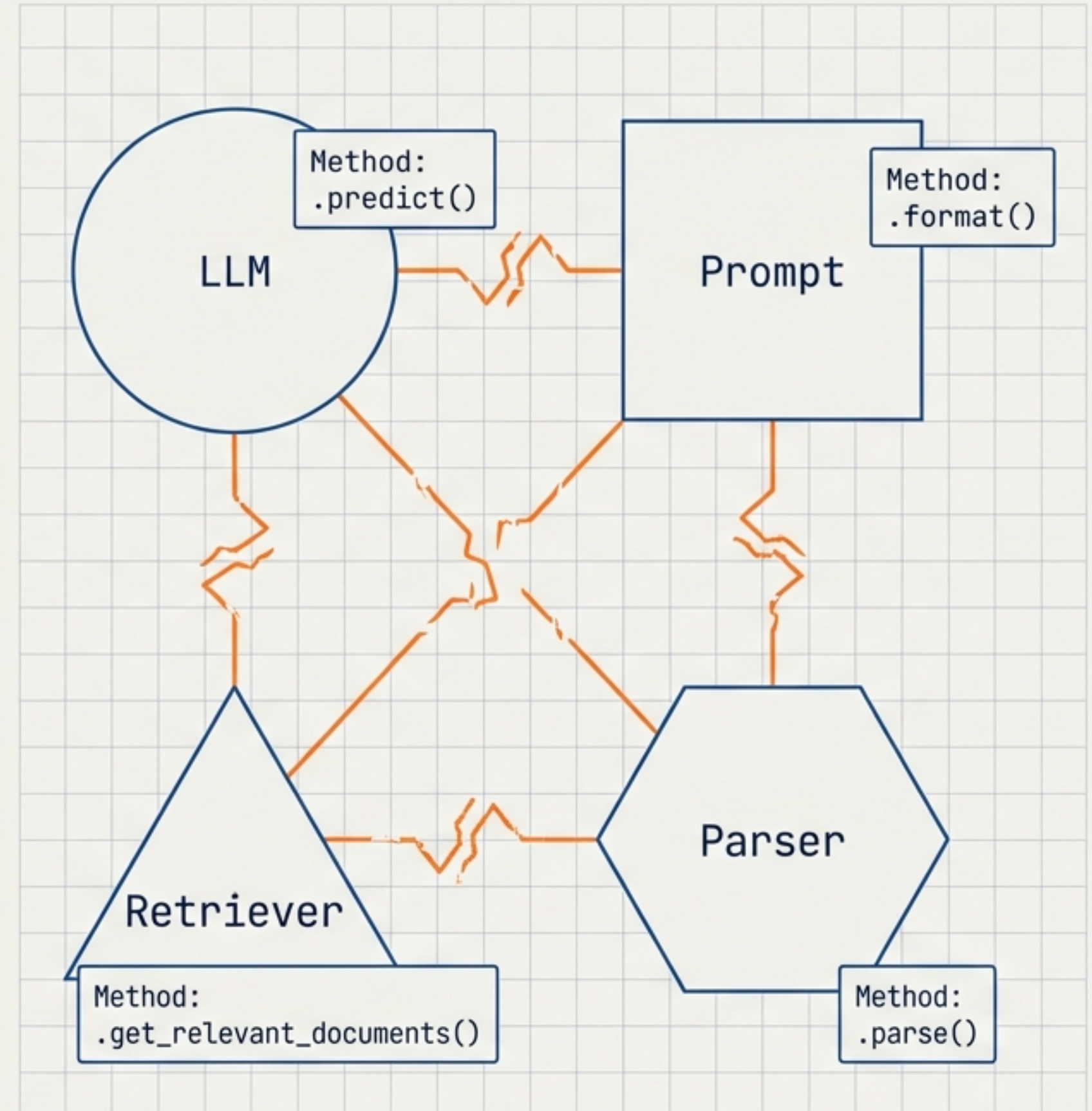
## The Trap: The “Too Many Chains” Problem

Anticipating every use case led to a hardcoded Chain class for each one. The library swelled with over 50 specific types, creating a massive maintenance burden and a steep learning curve.



# Root Cause Analysis: Lack of Standardisation

The core issue: Components were not designed to speak to each other. They used disparate methods, requiring custom “glue code” for every connection.



# The Solution: The Runnable Protocol



A **Runnable** is a unit of work with a guaranteed, standard interface.

The Shift: Instead of custom glue code, every component (Prompts, Models, Parsers) was refactored to speak the exact same language. If everything is a Runnable, everything connects automatically.

# The Standard Interface: The "Invoke" Era

**``invoke(input)``**

Process a single input and return an output.

```
model.invoke("Hello")
```

**``batch([inputs])``**

Process a list of inputs in parallel.

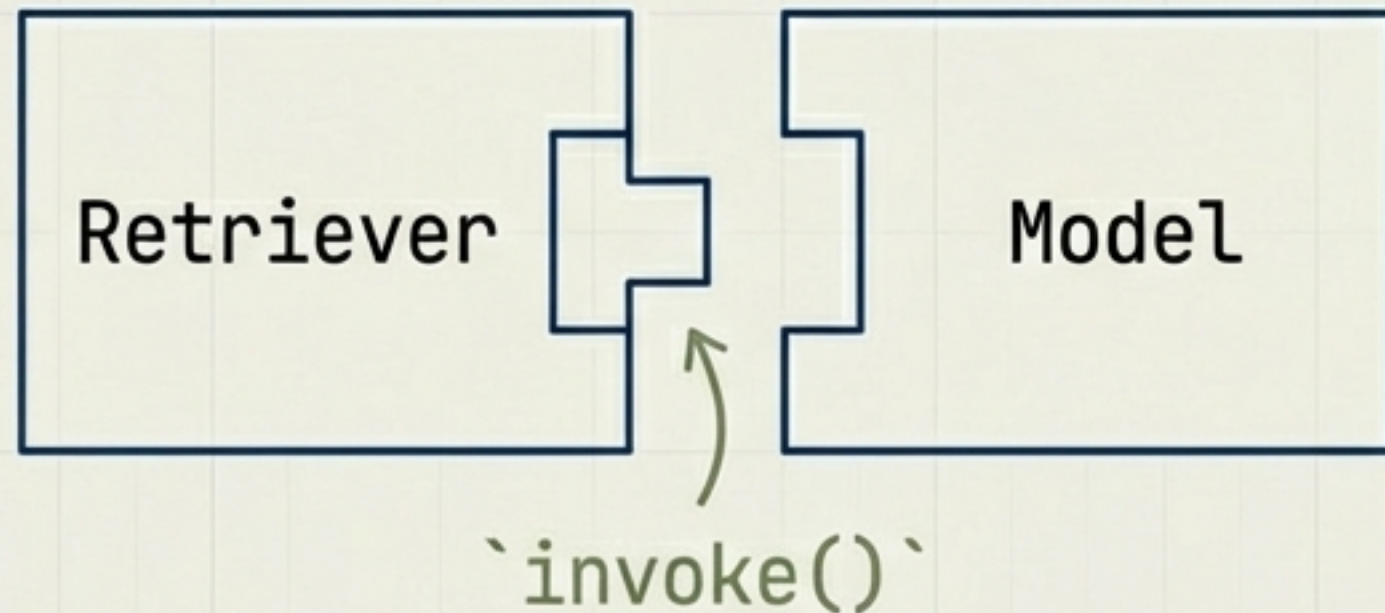
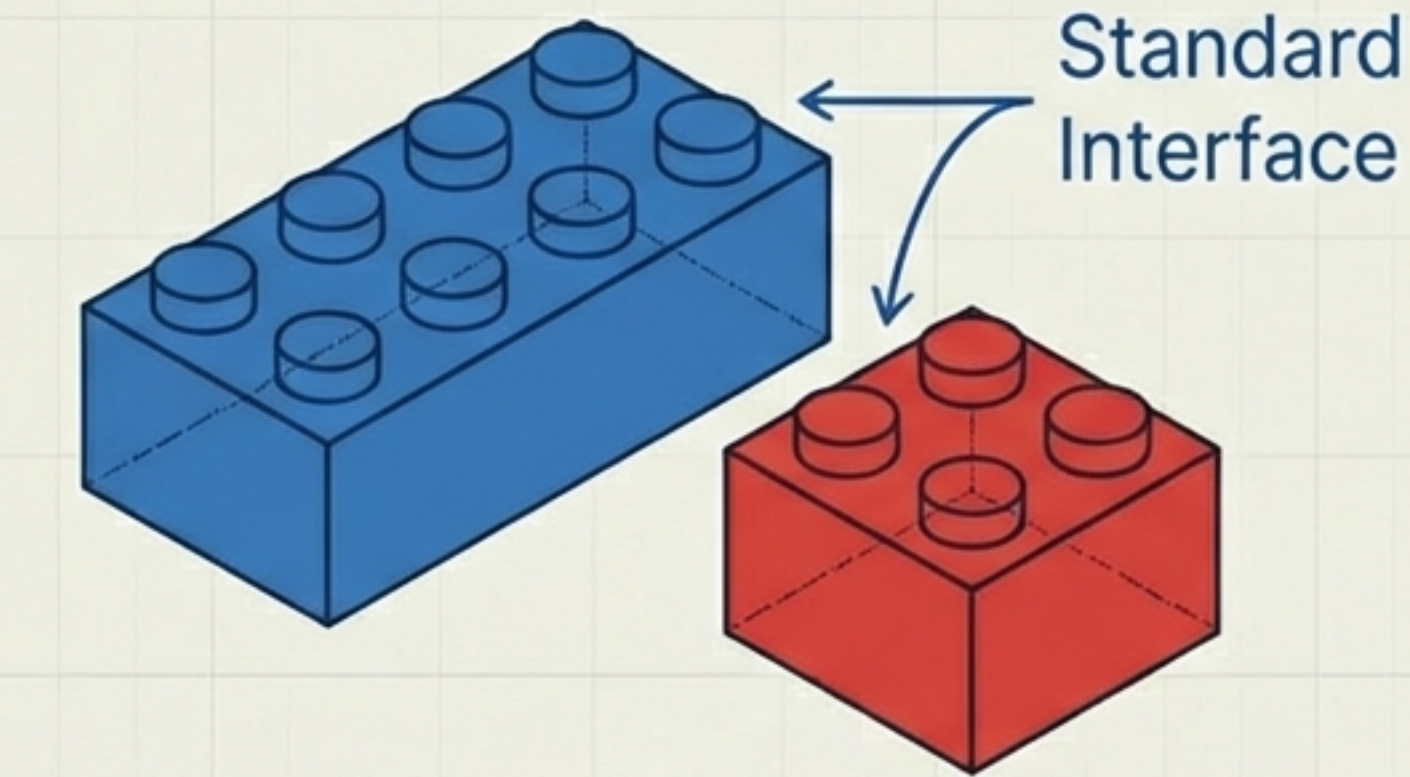
```
model.batch(["Hello", "Hi"])
```

**``stream(input)``**

Stream output chunks for real-time responses.

```
for chunk in model.stream("Hello"): print(chunk)
```

## Architectural Blueprint & Clean Engineering



## The Lego Analogy

### Standardisation Unlocks Composability

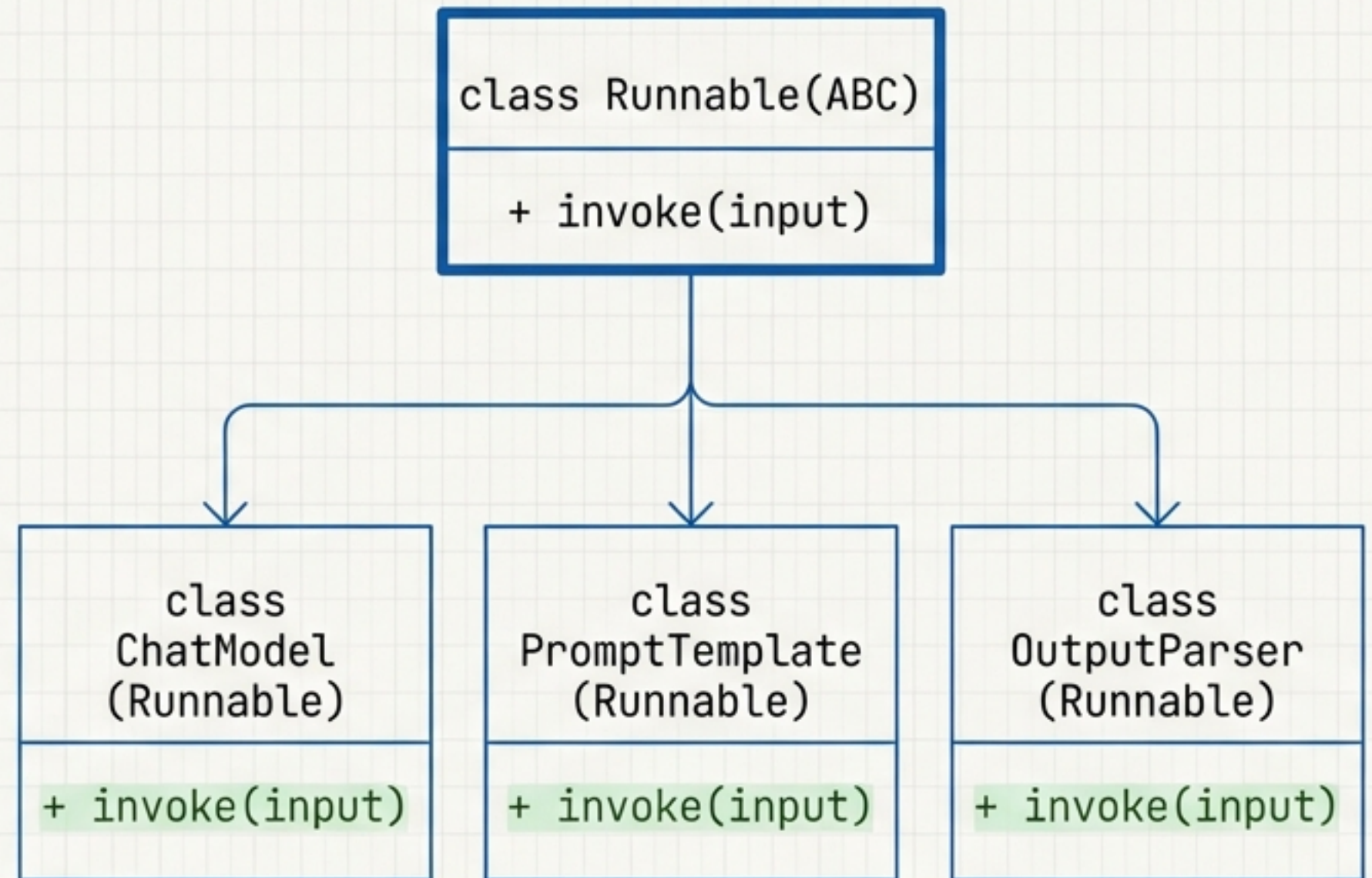
- **Lego Blocks:** Different shapes and purposes, but they all share the exact same connecting stud.
- **Runnables:** Different logic (Models, Retrievers, Parsers), but they all share the `invoke` interface.
- **Result:** Standardization enables infinite composability without custom glue code.

# Under the Hood: Python Abstract Base Classes

How is this enforced? Through Python's **ABC (Abstract Base Class)** system.

1. Define a parent class `Runnable(ABC)`.
2. Define an abstract method `invoke()`.
3. Force all child components to inherit from `Runnable`.

This strictly enforces the protocol; components without `invoke` will fail to compile.



# Adapting Components to the Protocol

Existing logic is wrapped into the new interface.

## ChatModel

```
class ChatModel(Runnable):  
    def invoke(self, input):  
        # Wraps old API  
        return self.client.predict(input)
```

## PromptTemplate

```
class PromptTemplate(Runnable):  
    def invoke(self, input):  
        # Wraps string logic  
        return self.template.format(**input)
```

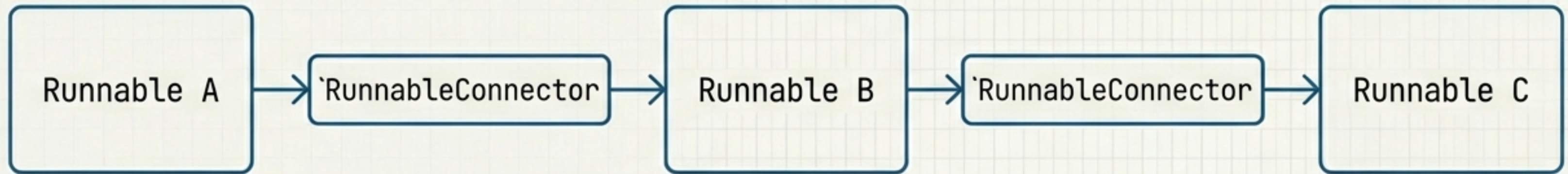
## OutputParser

```
class OutputParser(Runnable):  
    def invoke(self, input):  
        # Wraps parsing logic  
        return parse_json(input)
```

# The Power of Composition: The `RunnableConnector`

Because components share an interface, we can create a generic connector.

“Chains” are no longer hardcoded classes; they are dynamic lists of Runnables.

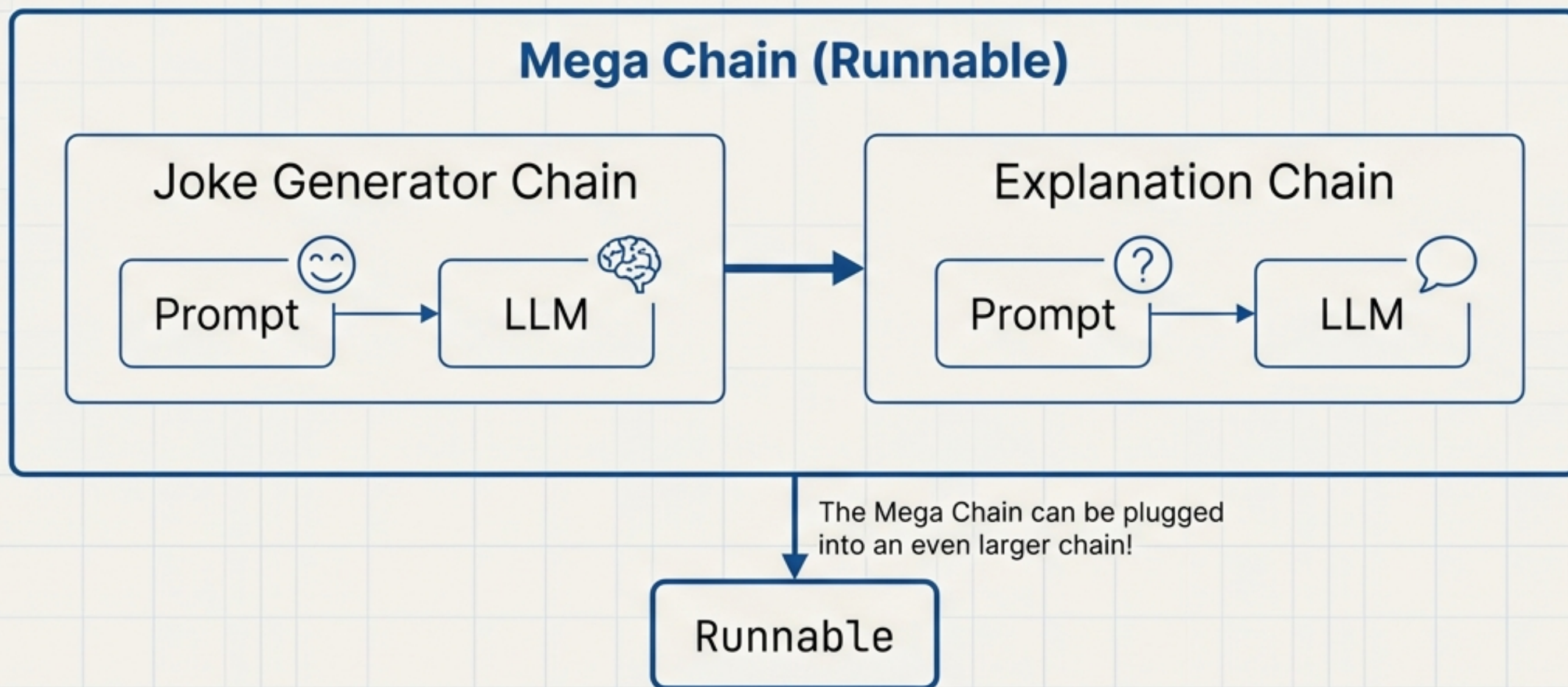


Output(A) == Input(B)

Output(B) == Input(C)

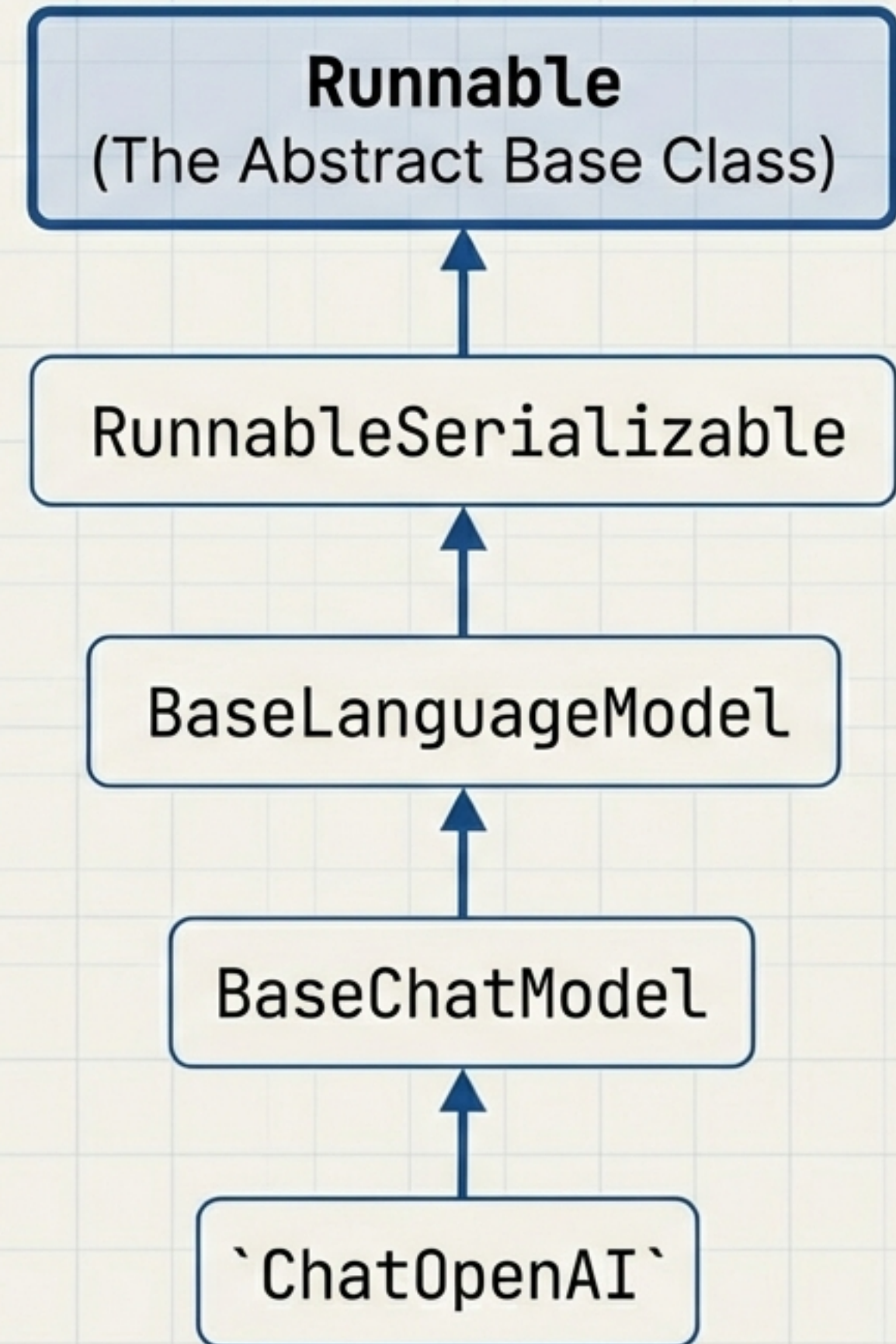
# Recursive Composition: Chains of Chains

A sequence of Runnables is also a Runnable. You can plug two separate chains together to create a "Mega Chain".



# Evidence in the Source Code

Inspecting the inheritance hierarchy of `ChatOpenAI` confirms that the entire library was re-architected around this single base class.



# Summary: The Physics of the LangChain Universe

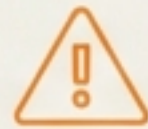
## The Old Way

- Hardcoded classes (`LLMChain`)

- Disparate interfaces (`.predict`, `.format`)



- Difficult to maintain and learn

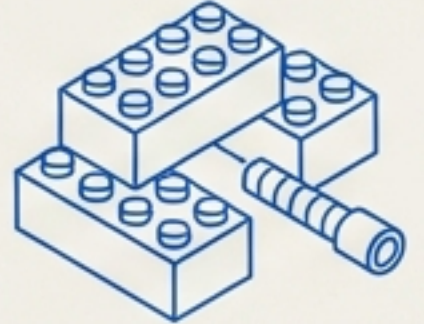


## The Runnable Way

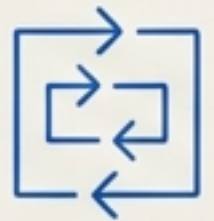
- Standard `invoke` protocol



- Universal composability (Lego blocks)



- Recursive and modular



Understanding Runnables is understanding how modular AI applications are built.